Préambule

Le TD3 explique des notions extrêmement importantes. Il aborde :

- Structure de données
- Types abstraits de données
- (La complexité en moyenne/la complexité amorti) facultatif
- La notion de complexité
- Les structures linéaire
- Les class python
- Les listes chainées
- Les listes doublement chainées
- Les piles
- Les files
- Les arbres
- Les parcours d'arbre

Il est clairement impossible d'expliquer tout ça à la volé et que tout le monde comprenne.
Je mets mes corrections des exos. Ils sont là pour mieux comprendre.

Structure de données et types abstraits de données

L'objectif est de stocker des données. On a plusieurs opérations importantes :

- Stocker
- Accéder
- Modifier

Un **type abstrait de données** donne les spécifications de la structure. Par exemple pour une liste on va demander :

- Pouvoir accéder à l'aide d'un indice `liste[n]`
- Pouvoir modifier une case du tableau
- Pouvoir ajouter un élément dans le tableau
- Pouvoir supprimer un élément du tableau

C'est les opérations qu'elle peut effectuer.
Dans les types abstrait de données que nous allons voir nous avons : les listes, les files, les piles, les listes chainées, les listes doublement chainées.

A l'inverse la **structure de données** est l'implémentation du type abstrait de données. Par exemple, cette parti explique comment on supprime un élément de la liste. C'est à ce moment que l'on peut spécifier le coût de chaque action. En fonction, de la structure de données une action du type abstrait de données peut avoir un coût plus ou moins grand.

Coût/complexité en moyenne et coût/complexité amorti

Cette partie est en gris dans le cours. Elle est donc facultative mais cela reste des notions importantes d'algorithmie.

Un algorithme peut avoir une très mauvaise complexité dans le pire cas mais être efficace en pratique (par exemple le tri fusion). Pour montrer cela on utilise les notions de **coût en moyenne** et de **coût amorti**. Voilà les définitions données dans le cours

- **En moyenne**: coût moyen d'une opération, calculé sur les différentes situations possibles, selon un modèle ou une distribution des cas
  Tient compte de la variabilité des situations possibles et du modèle probabiliste choisi
- **Amorti** : répartition du coût des opérations exceptionnelles et coûteuses sur un grand nombre d'opérations courantes et peu coûteuses
  Pour bien comprendre des fonctions

Pour une meilleur compréhension voilà un cas pour les 2 notions.

Pour le coût en moyenne. Prenons l'exemple du tri rapide qui tri une liste (je ne vais pas expliquer le principe du tri rapide ni son implémentation mais c'est intéressant). Dans notre cas nous prenons une liste d'entiers allant de 1 à n et rangés de façon aléatoire. Il faut définir une distribution (une loi de probabilité) sur les données en entré. Le plus souvent c'est la distribution uniforme. Chaque liste a la même chance d'arrivée. La complexité dans le meilleur cas est en $O(nlog(n))$ (log en base 2 on est en informatique). La complexité dans le pire cas est en $O(n^2)$ et en moyenne si on considère l'ensemble des listes possibles et si on choisi la distribution uniforme on est en $O(nlog(n))$. Les cas qui sont en $O(n^2)$ sont très rares. Il est souvent difficile de réaliser le coût en moyenne d'un algorithme.

Pour le coût amorti. Dans le principe du coût on essaye de majorer le coût d'une suite d'opération et on regarde le coût de la somme des opérations divisées par le nombre d'opération effectuer. Par exemple, on a une cave et on rentre des colis à l'intérieur et on souhaite avoir tout les colis dans une unique cave. Ajouter un colis se fait en temps constant. Mais un moment la cave peut être entièrement rempli. A ce moment il faut déplacer l'ensemble des colis dans une cave plus grande. La complexité de cette opération est en $O(n)$. Mais celle-ci est rare. Ainsi, la complexité d'ajouter d'un colis peut-être en $O(n)$ dans le pire cas mais sa complexité amorti n'est qu'un $O(1)$.

D'un point vue générale, en moyenne on définie une loi de probabilité sur les données d'entrées et on analyse le cas.
En complexité amorti on essaye de majorer le coût de n opérations. Après on divisera le coût de ces n opérations par n.

Les structures linéaires

Je n'ai rien à rajouter. Je peux expliquer plus précisément certaine complexité si vous le voulez.

Les classes python

Le but d'une `class` en python est de définir de nouveau objet. Chaque objet possède ces propres variables et ces propres fonctions.
Tout d'abord, pour pouvoir interagir avec notre objet il faut l'initialiser. L'initialisation se fait par la fonction `def __init__(self):`. Par exemple, pour un étudiant pour l'initialiser il faut faire cela :

```
class Etudiant:
    def __init__(self, nom, groupe):
        self.nom = nom
        self.groupe = groupe

eleve1 = Etudiant("Le Picard", "G1")
```

Ainsi un étudiant possède 2 variables : `nom` et `groupe`. Pour accéder à une de ses variables, il faut faire :

```
print(eleve1.nom)
print(eleve1.groupe)
```

Mais on peut également définir des fonction dans la `class`. On écrit une fonction comme d'habitude mais là on le fait dans la `class`. Pour y accéder c'est la même façon que pour les variables. Attention, lors de la création de la fonction, il faut **obligatoirement mettre l'argument** `self` parmi les argument de la fonction , mais lors de l'appel de la fonction il ne faut pas donné cette argument. Il est déjà connu par la machine. Par exemple :

```
class Etudiant:
    def __init__(self, nom, groupe):
        self.nom = nom
        self.groupe = groupe

    def affiche_information(self):
```

```
        print("Le nom est " + self.nom + " et son groupe est " + self.groupe)

    def change_groupe(self, nouveau_groupe): # self est obligatoire
        self.groupe = nouveau_groupe
        print("Le groupe a été changé")

eleve1 = Etudiant("Le Picard", "G1")
eleve1.affiche_information()
eleve1.change_groupe("G2")
eleve1.affiche_information()
```

On peut définir des fonctions plus spécifique. On a tout d'abord la méthode statique. Il faut rajouter `@staticmethod` avant la fonction. C'est une fonction qui n'a pas besoin de `self`. La deuxième fonction spécifique est la méthode de classe. C'est une fonction qui va s'appuyer sur les variables de classe. Les variables de classes sont des variables à tout les objets d'une même classe. Pour faire cette fonction il faut ajouter `@classmethod` et il faut **obligatoire mettre** `cls` parmi les argument de la fonction. Par exemple:

```
class Etudiant: # La majuscule est obligatoire
    nombre_etudiant = 0 # on initialise la variable de class avec cette ligne

    @classmethod
    def afficher_nb_etudiants(cls): # cls est obligatoire pour classmethod
        print(cls.nombre_etudiant)

    def __init__(self, nom, groupe):
        self.nom = nom
        self.groupe = groupe
        Etudiant.nombre_etudiant += 1 # Modifie la variable de class
nombre_etudiant

    @staticmethod
    def print_phrase():
        print("J'adore les cours d'AAP")

eleve1 = Etudiant("Le Picard", "G1")
eleve2 = Etudiant("Bachelet", "G2")
eleve1.afficher_nb_etudiants() # Renvoie 2
eleve2.afficher_nb_etudiants() # Renvoie également 2, nombre_etudiant est une
variable commune à eleve1 et eleve2
eleve1.print_phrase()
```

Fonction locale

Dans une même fonction on peut définir une fonction locale. C'est comme une variable locale. Elle n'est accessible que depuis cette fonction. Cela permet de faire des fonctions qui ne sont utiles que pour une fonction plus grande et ainsi améliorer la lisibilité de certains codes. Par exemple :

```
class Maclass:
    def __init__(self):
        pass

    def foo(self,liste):
        def fonction_auxiliaire(liste_aux, indice):
            if indice < len(liste_aux):
                return liste_aux[indice]
            else:
                return None
        print(fonction_auxiliaire(liste,0))
        print(fonction_auxiliaire(liste,1))
        print(fonction_auxiliaire(liste,17))

objet = Maclass()
objet.foo([0,1])
```

Récursivité

La récursivité consiste à faire une fonction qui s'appelle elle-même. C'est comme les récurrences en mathématiques. Par exemple voilà l'exponentiation rapide :

```
def expo(v, n):
    ''' La fonction renvoie v à la puissance n avec n un entier naturel'''
    if n == 0:
        return 1
    elif n % 2 == 0:
        tmp = expo(v,n//2)
        return tmp * tmp
    else:
        tmp = expo(v, (n-1)//2)
        return tmp * tmp * v

print(expo(2,16))
```

Savoir utiliser la récursivité est vraiment important

En plus la récursivité c'est beau

Liste chainée

Une liste chainée est un ensemble de chainons. Chaque chainon est constitué de 2 éléments. Le premier élément est une valeur stocker. Le deuxième élément est la suite de la liste (d'où le nom *liste chainée*). Comme le deuxième élément pointe lui aussi vers une liste chainée on parle de structure auto-référente (une structure qui fait appel à sa propre structure).
Pour réaliser cela on a besoin de créer la `class Cellule` :

```
class Cellule:
    def __init__(self,valeur,suiv=None):
        self.valeur = valeur
        self.suivant = suiv
```

Pour initier une liste chainée on fait :

```
liste_chaine = Cellule(0,None)
# Si on veut réaliser une chaine on fait
liste_chaine = Cellule(1,liste_chaine)
# On obtient Cellule(1, Cellule(0,None))
print(liste_chaine.valeur, liste_chaine.suivant.valeur)
```

Le problème de cette façon de procéder est qu'on ne peut pas créer de listes chainées vide. Quand on crée une liste chainée avec la structure de données actuelle on est obligé de donner une valeur pour créer un chainon. Pour résoudre ce problème on crée une classe qui va utiliser la `class chainon`. On obtient alors :

```
class Cellule:
    def __init__(self,valeur,suiv=None):
        self.valeur = valeur
        self.suivant = suiv

class ListeChainee:
    def __init__(self):
        self.debut = None

liste_chaine = ListeChainee() # Là on a une liste chainée vide
liste_chaine.debut = Cellule(0, liste_chaine.debut)
liste_chaine.debut = Cellule(1, liste_chaine.debut)
# Cela réalise la même liste chainée qu'avant

print(liste_chaine.debut.valeur, liste_chaine.debut.suivant.valeur)
```

Liste doublement chainée

C'est plus complexe. Je ne vais pas trop détailler.
Pour une liste de chainée, chaque chainon possède la suite de la liste. On ne peut pas revenir en arrière. La liste doublement chainée permet de résoudre ça. Chaque chainon possède l'élément qui le précède et l'élément qui le succède. De plus, on a aussi accès au début et à la fin de la liste.

Pile et file

La pile fonctionne comme une pile d'assiette. On empile les assiettes une par une. Pour enlever la première assiette mise, il faut enlever toutes celles au-dessus. Elle suit le principe LIFO (last in, first out)
La file fonctionne comme une file d'attente. Le premier arrivé est le premier à partir. Elle suit le principe de FIFO (first in, first out)

Les arbres

Un arbre est un type abstrait de données dans lequel on a des nœuds qui sont hiérarchisés. En haut, on a la racine. Ces arbres ressemblent beaucoup aux arbres généalogiques. On utilise le même vocabulaire. Ainsi chaque noeud peut avoir des fils et chaque noeud a un père (sauf la racine).

Un parcours consiste à parcourir l'ensemble des éléments d'un arbre et cela une unique fois. Il existe 2 grands types de parcours.

Le parcours en profondeur. On descend le plus profondément possible avant de revenir en arrière et explorer les différentes branches de l'arbre. C'est un parcours qui s'écrit facilement en récursif. Je vous conseille le récursif, cela va vous de bons automatismes pour les arbres.

Le parcours en largeur. On explore la racine, puis les descends de la racine, puis les descends des descends de la racine et ainsi de suite. On explore l'arbre niveau par niveau. L'utilisation d'une file est quasiment obligatoire pour écrire un parcours en largeur.

Annexe

Je peux expliquer ce que la récursivité terminale et comment rendre un algorithme récursif en algorithme itératif si vous voulez mais c'est clairement du hors programme